

2025“启策”杯有人无人协同 空中智能算法创新竞赛

AI 决策算法框架使用手册

大赛组委会

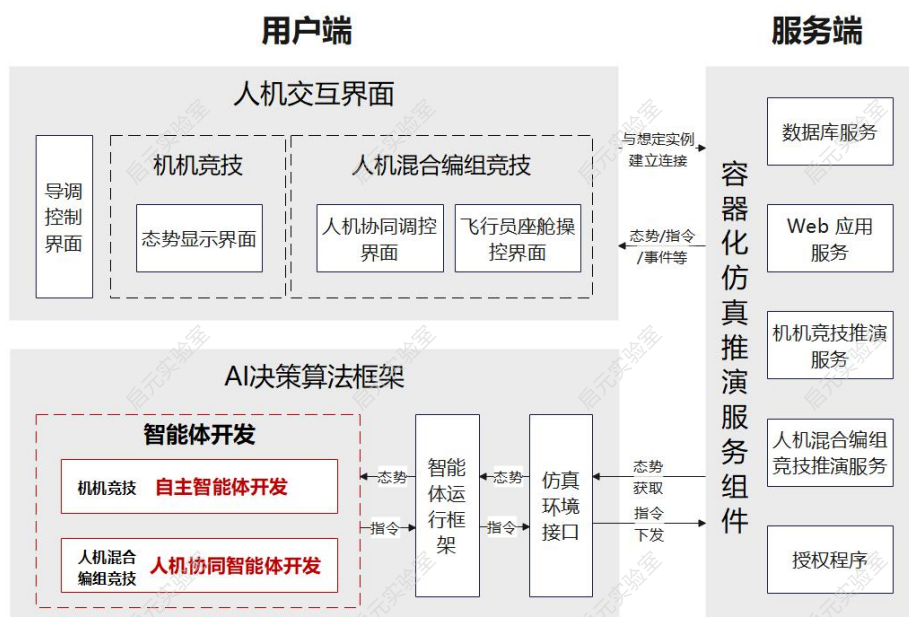
目录

1. AI 决策算法框架介绍	1
2. 仿真环境接口	2
2.1. 态势数据输入接口介绍	2
2.1.1. 仿真推演头信息	2
2.1.2. 双方阵营信息	3
2.2. 决策指令输出接口介绍	6
2.2.1. 到达指定速度指令	7
2.2.2. 线性加速到指定速度指令	7
2.2.3. 朝目标开火指令	7
2.2.4. 跟随目标机动指令	7
2.2.5. 朝目的地机动指令	8
2.2.6. 航线巡逻指令	8
2.2.7. 区域巡逻指令	8
3. 智能体运行框架	8
3.1. 配置文件	9
3.1.1. 机机竞技模式	9
3.1.2. 人机混合编组竞技模式配置	10
3.1.3. 选择竞技模式	11
3.2. 框架运行主逻辑	11
3.2.1. 程序运行入口	11
3.2.2. 框架运行流程	12
4. 智能体开发	13
4.1. 作品提交格式说明	13
4.2. 智能体开发流程	14
4.2.1. 机机竞技智能体开发	14
4.2.2. 人机混合编组竞技智能体开发	14
4.3. 智能体开发后验证配置	17

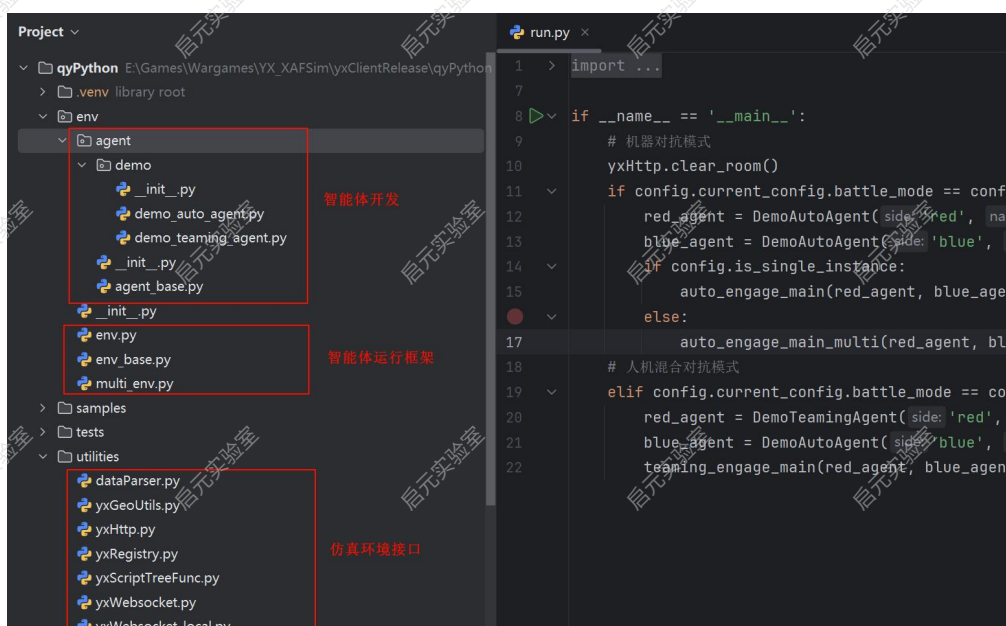
1. AI 决策算法框架介绍

本手册介绍如何借助 AI 决策算法框架开发智能体。

AI 决策算法框架是构成比赛开发环境套件组成部分。AI 决策算法框架通过仿真环境接口从服务端获取态势，并向服务器发送指令信息。在其内部设置智能体运行框架，可调度运行智能体进行态势感知与决策控制，并通过仿真环境接口进行信息交互。



AI 决策算法框架的源代码打包在 `qyPython_{xxxx}.zip` (其中，{xxxx}表示版本，{xxxx} ≥ 1201)。在 AI 决策算法框架中，仿真环境接口、智能体运行框架、智能体开发三部分的代码逻辑关系如下图所示：



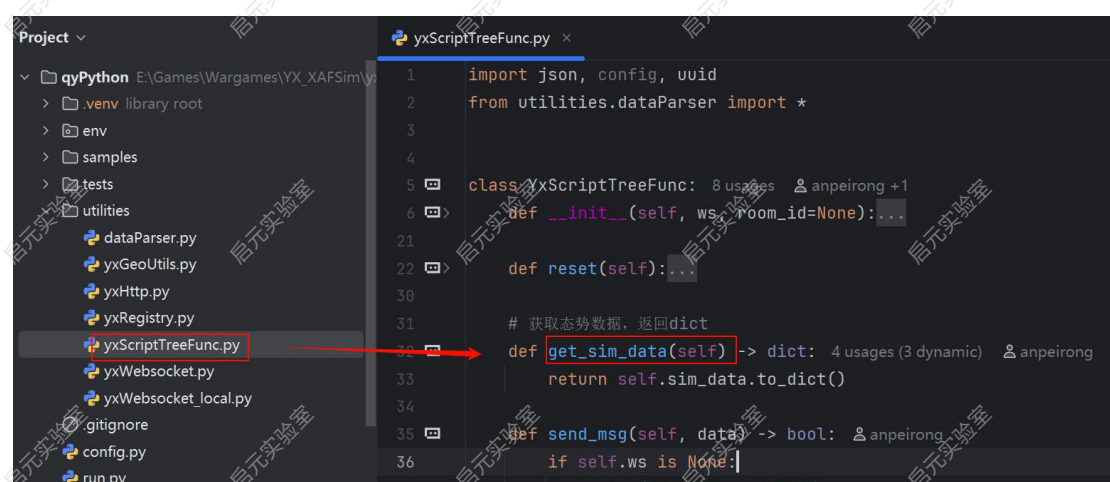
参赛用户可在智能体运行框架中开发用于机机竞技的机器自主智能体、以及用于人机混合编组竞技的人机协同智能体。

2. 仿真环境接口

仿真环境通过 `websocket` 建立起与服务端的连接，用于获取态势信息、推送决策指令。态势信息、决策指令经过结构化封装，统一由 `yxScriptTreeFunc.py` 提供。

2.1. 态势数据输入接口介绍

`yxScriptTreeFunc.py` 通过函数 `get_sim_data()` 返回态势数据字典。



态势数据字典包含两大部分：“header”仿真推演头信息、“side_list”双方阵营列表信息。



2.1.1. 仿真推演头信息

“header”头信息记录了仿真推演的基本信息。

键名	键的含义	单位	取值
sim_time	仿真时间	秒	浮点数
real_time	系统时间	秒	浮点数
is_active	是否在推演	/	True/False
time_ratio	时间加速倍率	/	正整数

```

"header": {
  "sim_time": 249.4,          # 仿真时间 (单位: 秒)
  "real_time": 1764492528.347569, # 系统时间 (单位: 秒)
  "is_active": True,          # 是否在推演中
  "time_ratio": 1             # 时间倍率
},                             # 仿真推演头信息

```

仿真推演头信息

2.1.2. 双方阵营信息

“side_list” 双方阵营信息，以数组形式分别记录了红方阵营、蓝方阵营的视角信息。

```

{
  "header": { ... },          # 仿真推演头信息
  "side_list": [
    {
      "side": "red",          # 红方阵营
      "platform_list": [...], # 红方现存平台信息
      "track_list": [...],    # 红方当前探测的单位信息
      "broken_list": [...],   # 红方已损失的单位信息
    },
    {
      "side": "blue",         # 蓝方阵营
      "platform_list": [...], # 蓝方现存平台信息
      "track_list": [...],    # 蓝方当前探测的单位信息
      "broken_list": [...],   # 蓝方已损失的单位信息
    }
  ]
}

```

红方阵营视角信息

蓝方阵营视角信息

双方阵营列表

某一方的阵营视角信息包括四种信息：

键名	键的含义	取值
side	阵营名称	“red”/“blue”
platform_list	该阵营当前现存平台信息	数组
track_list	该阵营当前探测到的单位信息	数组
broken_list	该阵营已损失的单位信息	数组

参赛用户在开发智能体时，只能获取到己方阵营视角信息，如下图所示。详情见下文「4.智能体开发」。

```
def update_decision(self, new_observation: Dict):
    """更新决策逻辑"""
    self.last_observation = self.observation
    self.observation = new_observation
    self.frame_count += 1 # 增加帧计
```

new_observation = {dict: 4} ('broken_list': ['红无人机8'], 'platform_list': [{'altitude': 2854.12, 'heading': 2.755017932236317, 'id': 1, 'is_co': 0, 'pitch': 0, 'roll': 0, 'side': (str) 'red', 'speed': 10, 'type': '无人机', 'velocity_x': 0, 'velocity_y': 0, 'velocity_z': 0, 'weapons/quantity': 0}], 'track_list': [(list: 3) [{'altitude': 3326.5642, 'engage_target': '蓝无人机7', 'heading': 1.0608}], 'broken_list' = (list: 1) ['红无人机8']

2.1.2.1. 阵营当前现存平台信息

“platform_list”，以数组形式记录了己方阵营当前现存平台信息。包含信息如下：

键名	键的含义	单位	取值
id	平台id	/	正整数
side	阵营方	/	“red”/“blue”
type	平台类型	/	“有人机”/“无人机”
speed	平台飞行速度	米/秒	正浮点数
name	平台名称	/	字符串
longitude	飞行经度	度	浮点数, (-180,180)
latitude	飞行纬度	度	浮点数, (-90,90)
altitude	飞行海拔	米	正浮点数
heading	偏航角	弧度	浮点数, $(-\pi, \pi)$. 0是正北, 顺时针为正, 逆时针为负
pitch	俯仰角	弧度	浮点数, $(-\pi/2, \pi/2)$. 0是平视, $-\pi/2$ 为下, $\pi/2$ 为上
roll	横滚角	弧度	浮点数, $(-\pi, \pi)$. 0是水平, 负值为左横滚, 正值为右横滚
velocity_x	速度x分量	米/秒	浮点数. 东为正, 西为负
velocity_y	速度y分量	米/秒	浮点数. 北为正, 南为负
velocity_z	速度z分量	米/秒	浮点数. 直向上为正, 垂直向下为负
weapons/quantity	平台挂载武器 剩余数量	枚	非负整数

```

"platform_list": [
  {
    "id": 1,
    "side": "red",
    "type": "无人机",
    "speed": 447.0441,
    "name": "红无人机1",
    "longitude": 145.9127479,
    "latitude": 33.4996353,
    "altitude": 3459.61,
    "heading": -2.998339202054858,
    "pitch": 0.0010105456369047167,
    "roll": -1.4009496106370685,
    "velocity_x": -440.50448810171736,
    "velocity_y": -76.18405462949177,
    "velocity_z": -0.4518698739544384,
    "is_controlled": None,
    "mover": {...},
    "sensors": [...],
    "weapons": [
      {
        "id": 5,
        "name": "导弹",
        "launched_platform_type": "导弹",
        "type": "WSF_EXPLICIT_WEAPON",
        "quantity": 2
      }
    ]
  }
]

```

2.1.2.2. 阵营当前探测到的单位信息

“track_list”，以数组形式记录了己方阵营当前探测到的单位信息，包括己方发射的导弹、敌方飞机平台、敌方发射的导弹。包含信息如下：

键名	键的含义	单位	取值
target_id	探测到单位id	/	正整数
target_name	探测到的单位名称	/	字符串
platform_entity_type	探测到的单位类型	/	“导弹”/“有人机”/“无人机”
platform_entity_side	探测到单位的阵营方	/	“red”/“blue”
longitude	探测到单位的飞行经度	度	浮点数, (-180,180)
latitude	探测到单位的飞行纬度	度	浮点数, (-90,90)
altitude	探测到单位的飞行海拔	米	正浮点数
speed	探测到单位的飞行速度	米/秒	正浮点数
v_x	探测到单位速度x分量	米/秒	浮点数. 东为正, 西为负
v_y	探测到单位速度y分量	米/秒	浮点数. 北为正, 南为负
v_z	探测到单位速度z分量	米/秒	浮点数. 上为正, 下为负
heading	探测到单位偏航角	弧度	浮点数, $(-\pi, \pi)$. 0是正北, 顺时针

			针为正，逆时针为负
pitch	探测到单位俯仰角	弧度	浮点数, $(-\pi/2, \pi/2)$. 0是平视, $-\pi/2$ 为下, $\pi/2$ 为上
roll	探测到单位横滚角	弧度	浮点数, $(-\pi, \pi)$. 0是水平, 负值为左横滚, 正值为右横滚
is_fired_num	该单位当前被我方导弹攻击的数量	枚	非负整数
engage_target	该单位交战的敌方目标	/	仅我方导弹单位有该属性

```

"back_list": [
  {
    "target_id": 281,
    "target_name": "红无人机1_导弹_4",
    "platform_entity_type": "导弹",
    "platform_entity_side": "red",
    "longitude": 146.0974,
    "latitude": 33.2581,
    "altitude": 3891.241,
    "speed": 1200,
    "v_x": -1129.1733935524214,
    "v_y": 406.0398080310301,
    "v_z": -9.95598249458385,
    "heading": 2.796399688801103,
    "pitch": 0.008297295263981043,
    "roll": 0,
    "is_fired_num": 0,
    "engage_target": "蓝无人机2"
  },
  { ... },
  { ... }
],
# 红方当前探测的单位信息

```

探测到单位id (我方发射的导弹、敌方平台、敌方发射的导弹)
 # 单位的名称
 # 单位的类型 (导弹/无人机/有人机)
 # 单位的所属方 (red/blue)
 # 单位的飞行经度 (单位: 度)
 # 单位的飞行纬度 (单位: 度)
 # 单位的飞行高度 (单位: 米)
 # 单位的飞行速度 (单位: 米/秒)
 # 单位的飞行速度x分量 (单位: 米/秒), 东为正, 西为负
 # 单位的飞行速度y分量 (单位: 米/秒), 北为正, 南为负
 # 单位的飞行速度z分量 (单位: 米/秒), 上为正, 下为负
 # 单位的偏航角 (单位: 弧度), 取值范围 $(-\pi, \pi)$, 0是正北, 顺时针为正, 逆时针为负
 # 单位的俯仰角 (单位: 弧度), 取值范围 $(-\pi/2, \pi/2)$, 0是平视, $-\pi/2$ 为下, $\pi/2$ 为上
 # 单位的横滚角 (单位: 弧度), 取值范围 $(-\pi, \pi)$, 0是水平, 负值为左滚, 正值为右滚
 # 该单位当前被我方导弹攻击的数量
 # 该单位交战的敌方目标 (仅我方导弹有该属性)

2.1.2.3. 阵营已损失单位信息

“broken_list”，以字符串数组形式记录了己方阵营已损失的单位信息，包括损失的有人机、无人机、导弹等。下图以红方阵营为例，展示了红方已损失的单位信息。

```

"broken_list": [
  "红无人机2_导弹_2",
  "红无人机1_导弹_1",
  "红无人机2",
]
# 红方已损失的单位信息

```

2.2. 决策指令输出接口介绍

yxScriptTreeFunc.py 通过静态方函数提供飞行平台的决策指令输出，共 7 个指令。调用决策指令时，可导入相关模块，如下所示：

```
from utilities.yxScriptTreeFunc import YxScriptTreeFunc as decCmd
```


2.2.1. 到达指定速度指令

- 指令: `go_to_speed(self_platform: str, speed: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `speed`: 速度 (单位: 米/秒)
- 使用示例: `decCmd.go_to_speed("红无人机1", 300)`

2.2.2. 线性加速到指定速度指令

- 指令: `go_to_speed_la(self_platform: str, speed: float, linear_accel: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `speed`: 速度 (单位: 米/秒)
 - `linear_accel`: 线性加速度 (单位: 米/秒²)
- 使用示例: `decCmd.go_to_speed_la("红无人机1", 300, 1)`

2.2.3. 朝目标开火指令

- 指令: `fire_track(self_platform: str, target_name: str)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `target_name`: 目标平台名称
- 使用示例: `decCmd.fire_track("红无人机1", "蓝无人机1")`

2.2.4. 跟随目标机动指令

- 指令: `fly_to_target(self_platform: str, target_platform: str, speed: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `target_platform`: 目标平台名称
 - `speed`: 速度 (单位: 米/秒)
- 使用示例: `decCmd.fly_to_target("红无人机1", "蓝无人机1", 400)`

2.2.5. 朝目的地机动指令

- 指令: `fly_to_point(self_platform: str, target_point: list, speed: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `target_point`: 目的地[纬度（度），经度（度），高度（米）]
 - `speed`: 速度（单位：米/秒）
- 使用示例: `decCmd.fly_to_point("红无人机1", [33.49, 145.91, 3000], 400)`

2.2.6. 航线巡逻指令

- 指令: `route_patrol(self_platform: str, points: list, speed: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `points`: 航线点列表 [[纬度, 经度, 海拔], [纬度, 经度, 海拔], ...]
 - `speed`: 速度（单位：米/秒）
- 使用示例:
`deCmd.route_patrol("红无人机1", [[33.49, 145.91, 3000], [33.49, 146.91, 3000]], 400)`

2.2.7. 区域巡逻指令

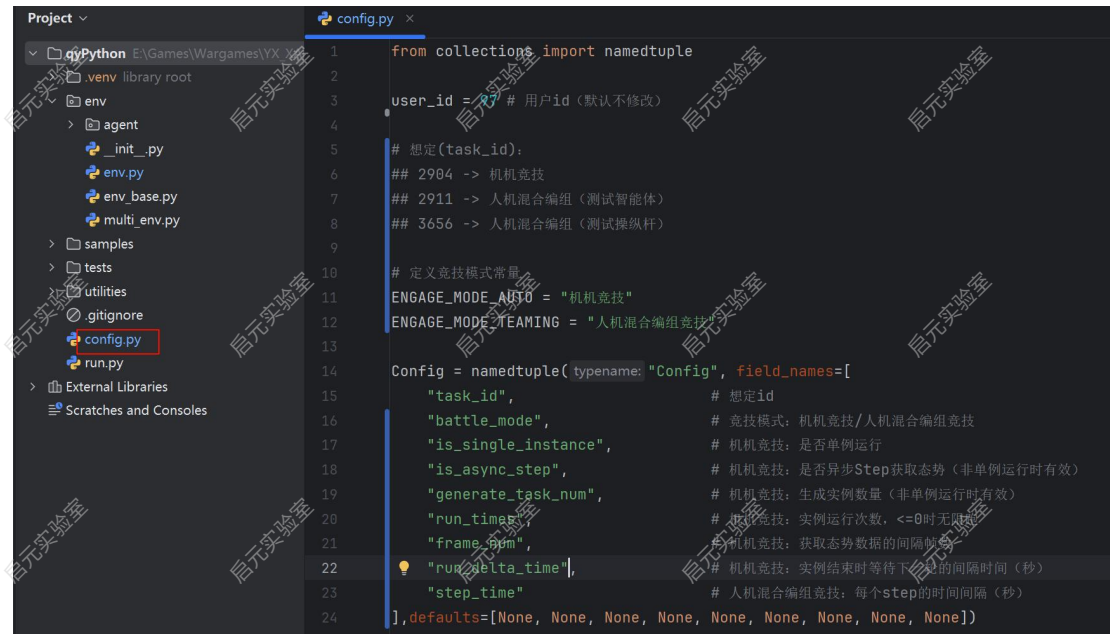
- 指令:
`area_patrol(self_platform: str, left_top_point: list, right_down_point: list, speed: float)`
- 输入参数:
 - `self_platform`: 自身平台名称
 - `left_top_point`: 左上角点位置 [纬度, 经度, 海拔]
 - `right_down_point`: 右下角点位置 [纬度, 经度, 海拔]
 - `speed`: 速度（单位：米/秒）
- 使用示例:
`deCmd.area_patrol("红无人机1", [34.49, 145.91, 3000], [33.49, 146.91, 3000], 400)`

3. 智能体运行框架

智能体运行框架可调度运行智能体进行态势感知与决策控制。

3.1. 配置文件

配置文件是 `config.py`，实现对机机竞技模式、人机混合编组竞技模式的配置，并选择程序要运行的竞技模式。



```
1 from collections import namedtuple
2
3 user_id = 29 # 用户id (默认不修改)
4
5 # 想定(task_id):
6 ## 2904 -> 机机竞技
7 ## 2911 -> 人机混合编组 (测试智能体)
8 ## 3656 -> 人机混合编组 (测试操纵杆)
9
10 # 定义竞技模式常量
11 ENGAGE_MODE_AUTO = "机机竞技"
12 ENGAGE_MODE_TEAMING = "人机混合编组竞技"
13
14 Config = namedtuple('Config', field_names=[
15     "task_id", # 想定id
16     "battle_mode", # 竞技模式: 机机竞技/人机混合编组竞技
17     "is_single_instance", # 机机竞技: 是否单例运行
18     "is_async_step", # 机机竞技: 是否异步Step获取态势 (非单例运行时有效)
19     "generate_task_num", # 机机竞技: 生成实例数量 (非单例运行时有效)
20     "run_times", # 机机竞技: 实例运行次数, <=0时无限
21     "frame_num", # 机机竞技: 获取态势数据的间隔帧数
22     "run_delta_time", # 机机竞技: 实例结束时等待下一轮的间隔时间 (秒)
23     "step_time", # 人机混合编组竞技: 每个step的时间间隔 (秒)
24 ], defaults=[None, None, None, None, None, None, None, None])
```

3.1.1. 机机竞技模式

机机竞技模式 (AUTO_ENGAGE_CONFIG) 可配置如下参数:

键名	键的含义	取值
is_single_instance	是否单例运行	True/False
is_async_step	是否异步Step获取态势 (非单例运行时有效)	True/False
generate_task_num	生成并发推演实例数量 (非单例运行时有效)	正整数
run_times	想定实例运行轮数. 当该值<=0时表示运行轮数无穷大.	整数
frame_num	获取态势数据的间隔帧数	正整数
run_delta_time	实例结束时等待下一轮运行的间隔时间 (秒)	正整数

在 `config.py` 中，配置参数项如下图所示：

```
# 机机竞技模式配置
AUTO_ENGAGE_CONFIG = Config(
    task_id=2904,
    battle_mode=ENGAGE_MODE_AUTO,
    is_single_instance=True,
    is_async_step=True,
    generate_task_num=2,
    run_times=3,
    frame_num=5,
    run_delta_time=5
)
```

机机竞技模式对应的推演想定 task_id 为 2904，在「导调控制界面」对应的想定名为「机机竞技模式」。



3.1.2. 人机混合编组竞技模式配置

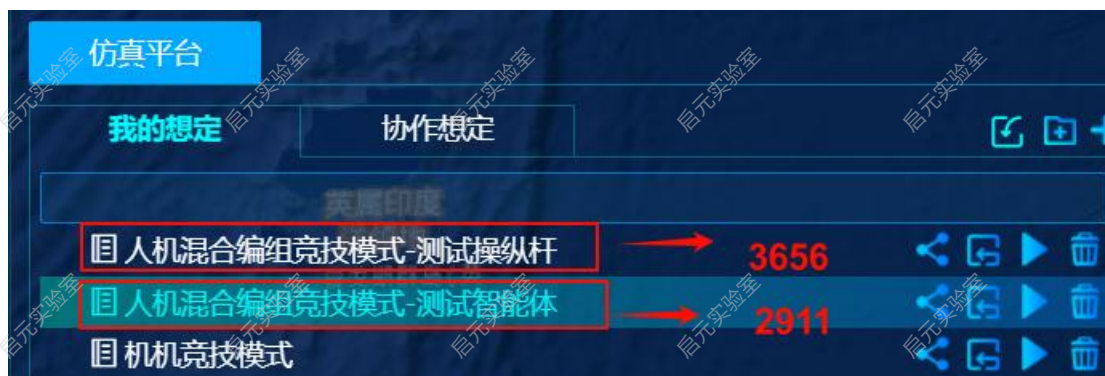
人机混合编组竞技模式（TEAMING_ENGAGE_CONFIG）可配置如下参数：

键名	键的含义	取值
task_id	推演想定id	2911/3656
step_time	每步推演step的时间间隔（秒）	正浮点数

在 config.py 中，配置参数项如下图所示：

```
# 人机混合编组竞技模式配置
TEAMING_ENGAGE_CONFIG = Config(
    task_id=2911, #3656
    battle_mode=ENGAGE_MODE_TEAMING,
    step_time=0.5
)
```

人机混合编组竞技模式对应的推演想定 task_id 为 2911 或 3656，在「导调控制界面」对应的想定名分别为「人机混合编组竞技模式-测试智能体」、「人机混合编组竞技模式-测试操纵杆」。



面向人机混合编组竞技的智能体开发时，优先考虑 `task_id` 设置为2911。这是因为「人机混合编组竞技模式-测试智能体」（`task_id=2911`）想定中，有人机平台预置了飞行巡逻路线，无需人员介入使用操纵杆控制有人机。参赛选手可以将专注于开发智能体，并作为副驾驶员在线调控无人机。

而在「人机混合编组竞技模式-测试操纵杆」（`task_id=3656`）想定中，红方有人机平台需要人员介入使用操纵杆控制。在开启AI 决策算法框架程序的前提下，红方阵营需要一人作为飞行员使用操纵杆控制有人机，一人作为副驾驶员调控无人机。

3.1.3. 选择竞技模式

在 `config.py` 中，配置参数 `current_config`，设置取值为：

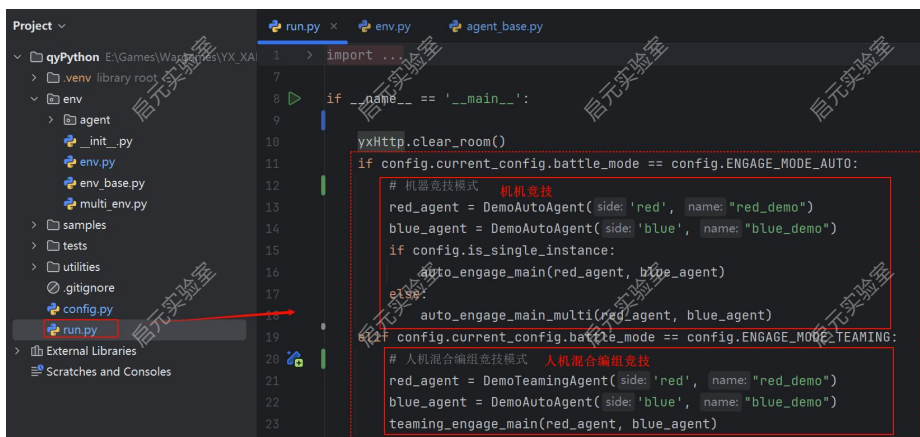
- `AUTO_ENGAGE_CONFIG`（机机竞技匹配）
- `TEAMING_ENGAGE_CONFIG`（人机混合编组竞技匹配）



3.2. 框架运行主逻辑

3.2.1. 程序运行入口

AI 决策算法框架的程序运行入口为 `run.py`，根据配置竞技模式选择相应的模式运行推演。



3.2.2. 框架运行流程

通过配置 config.py 文件设置，框架运行入口脚本 run.py 会执行不同的逻辑分支，实现机机竞技、人机混合编组竞技等推演。框架运行核心逻辑位于 env/env.py。框架运行流程的伪代码如下：

```

初始化环境(红方智能体, 蓝方智能体)
设置步长时间 = 配置中的步长时间
上一次执行时间 = 0

# 主循环
当 推演未结束:
    如果 仿真已开始:
        当前时间 = 获取仿真时间
        如果 当前时间 - 上一次执行时间 >= 步长时间:
            上一次执行时间 = 当前时间

            # 执行一步推演
            态势数据 = 获取当前仿真态势数据
            红方指令 = 红方智能体.执行决策(态势数据, 红方视角态势)
            蓝方指令 = 蓝方智能体.执行决策(态势数据, 蓝方视角态势)

            # 下发指令
            对于 每个指令:
                发送指令到仿真系统

            # 检查推演是否结束
            推演结束状态 = 检查推演结束条件()

# 推演结束处理
输出推演结果
停止仿真

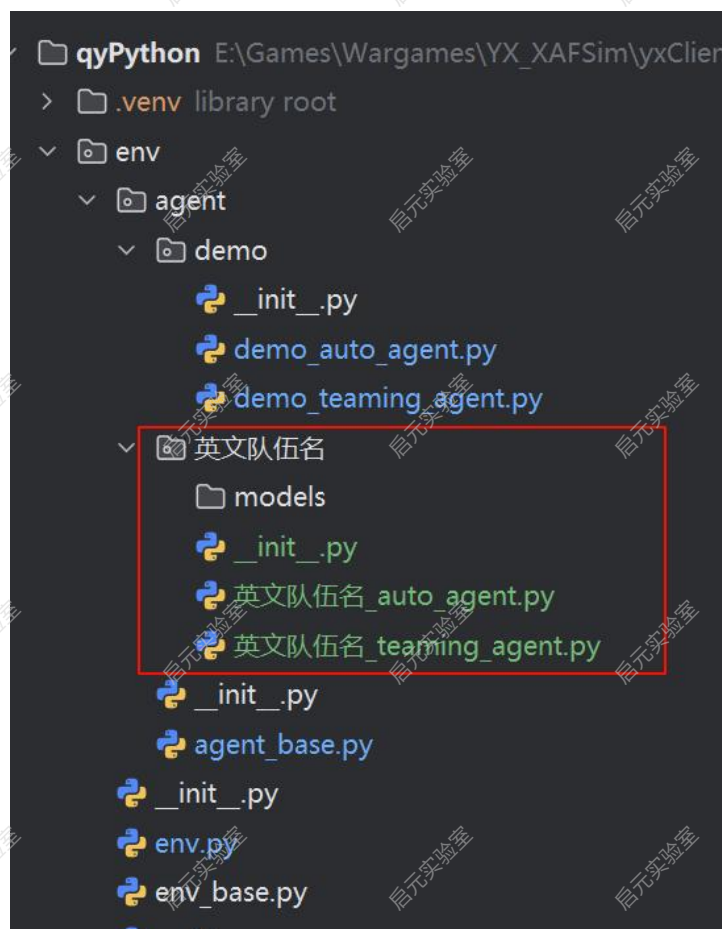
```

4. 智能体开发

4.1. 作品提交格式说明

参赛队伍提交作品文件格式如下所示，以 zip 包形式发送组委会官网：

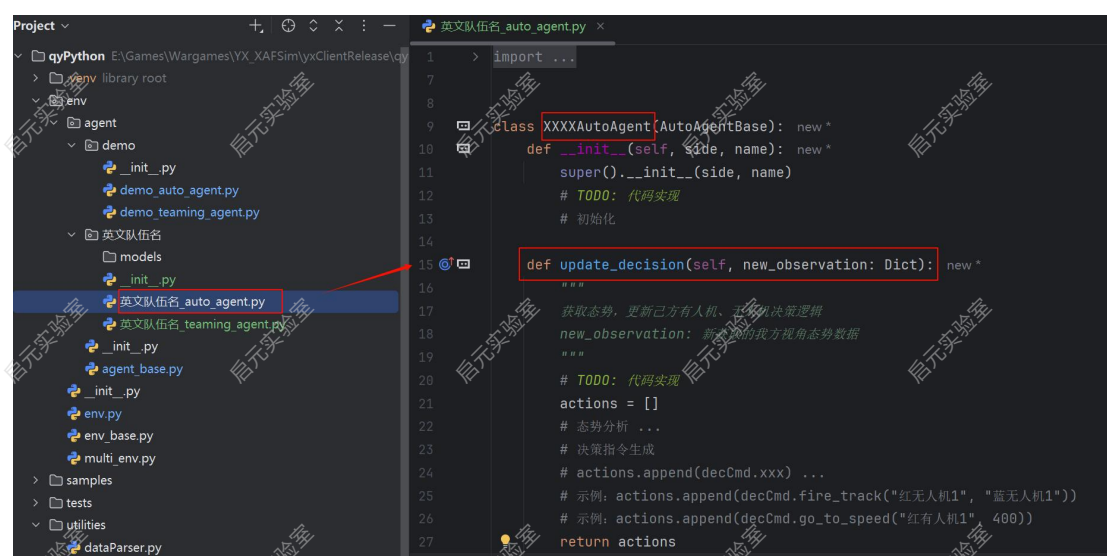
- 「英文队伍名」文件夹：替换成参赛队伍自定义的英文名
- models 文件夹：若参赛选手开发了 pytorch 模型，可将该模型（模型后缀.pkl）放置于该文件夹。
- 「英文队伍名_auto_agent.py」：开发用于机机竞技的机器自主智能体。文件名中中文替换成参赛队伍自定义的英文名。**机器自主智能体需要选手在初赛阶段提交，经过作品角逐优秀队伍进入复赛。**
- 「英文队伍名_teaming_agent.py」：开发用于人机混合编组竞技的人机协同智能体。文件名中中文替换成参赛队伍自定义的英文名。**人机协同智能体只在选手进入复赛阶段提交。**



4.2. 智能体开发流程

4.2.1. 机机竞技智能体开发

机机竞技智能体起名「XXXXAutoAgent」类，其中「XXXX」替换成参赛队伍自定义名称。「XXXXAutoAgent」类继承自 `AutoAgentBase` 类，需要实现 `update_decision` 函数功能：即获取态势 `new_observation`，进行态势分析，借助 `decCmd` 模块生成决策指令，用于己方有人机、无人机的决策控制。



开发用于机机竞技的智能体时,可以参考AI决策开发框架自带的基准智能体 `env/agent/demo/demo_auto_agent.py`。

⚠注意: 智能体开发时,可借鉴参考基准智能体的解析态势数据函数 `parse_observation` 的实现逻辑,但严禁抄袭、复用基准智能体用于态势分析的其他代码。

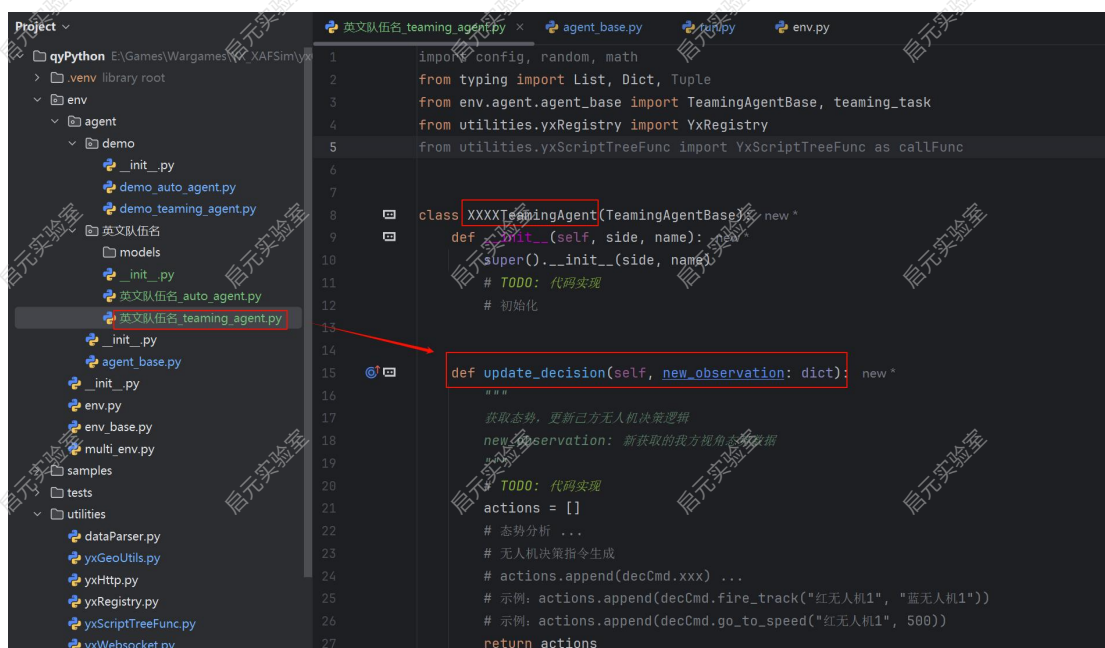
4.2.2. 人机混合编组竞技智能体开发

人机混合编组竞技智能体起名「XXXXTeamingAgent」类，其中「XXXX」替换成参赛队伍自定义名称。「XXXXTeamingAgent」类继承自 `TeamingAgentBase`类，有两类决策函数用于控制无人机：自主任务决策函数、有人介入决策函数。

4.2.2.1. 自主任务决策函数

自主任务决策函数是「XXXXTeamingAgent」类的 `update_decision` 函数：即获取态势 `new_observation`，进行态势分析，借助 `decCmd` 模块生成决策指令，用于己方无人机的自主决策控制。该函数逻辑可以复用机机竞技智能体的

update_decision 函数实现逻辑，也可以开发新的实现逻辑。



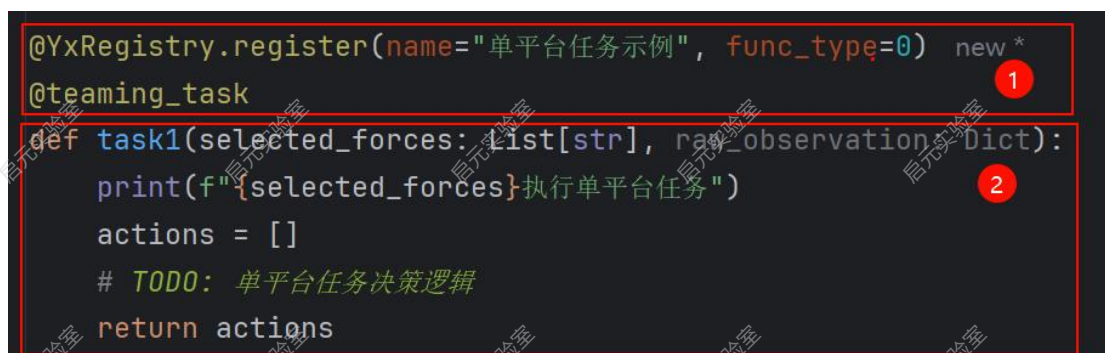
4.2.2.2. 有人介入决策函数

可在「英文队伍名_teaming_agent.py」文件中开发一到多个有人介入决策函数，可执行临机任务，实现人员对无人机的介入控制。

有人介入决策函数的模版如下图所示，函数上方必须带有两个装饰器 @YxRegistry.register、@teaming_task。

@YxRegistry.register 中参数介绍：

- name: 用户自定义，用于显示在副驾驶页面上的有人介入任务名称。
- func_type: 0 表示该函数处理「单平台任务」；1表示该函数处理「编组任务」；2表示该函数即可处理「单平台任务」，也可处理「编组任务」。



函数体部分，如下图所示，包括几个部分：

- 函数名：用户自定义，如“task1”
- selected_forces: 用于执行当前任务的我方平台列表，如["红有人机1", "

红无人机2"]

- raw_observation: 方视角态势信息，同 update_decision() 函数中态势信息

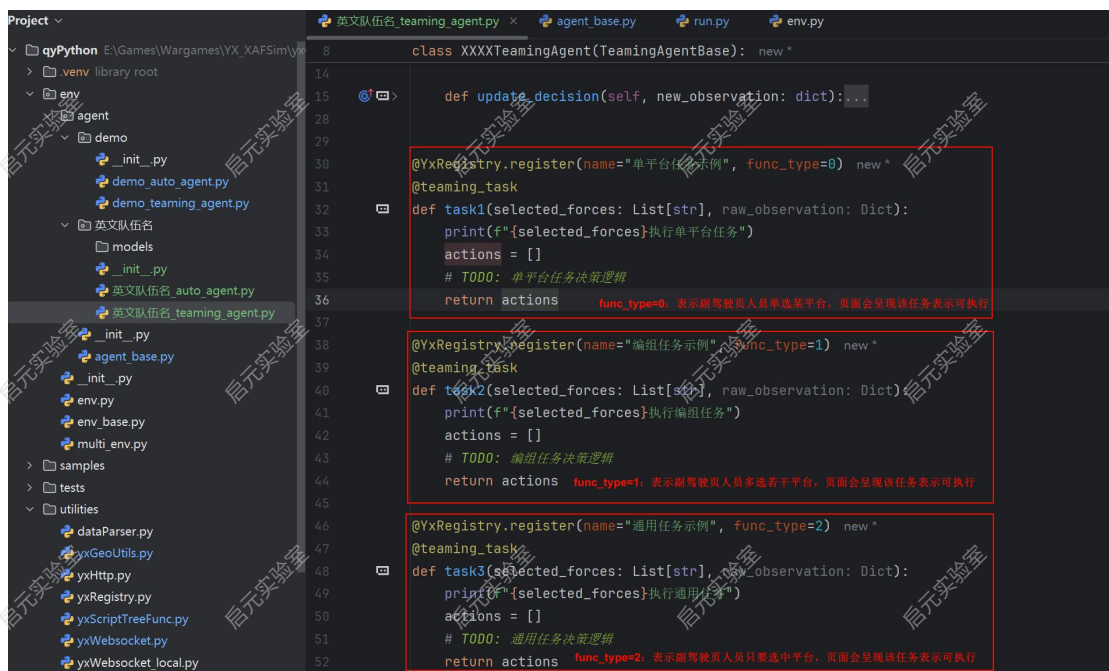
用户结合selected_forces、raw_observation信息，实现特定决策逻辑，对selected_forces执行有人介入任务。

```
@YxRegistry.register(name="单平台任务示例", func_type=0) new *
@teaming_task
def task1(selected_forces: List[str], raw_observation: Dict):
    print(f"{selected_forces}执行单平台任务")
    actions = []
    # TODO: 单平台任务决策逻辑
    return actions
```

4.2.2.2.1. 示例

在代码中开发了三个有人介入决策函数：

- task1: 表示副驾驶页人员单选某平台，页面会呈现该任务表示可执行
- task2: 表示副驾驶页人员多选若干平台，页面会呈现该任务表示可执行
- task3: 表示副驾驶页人员只要选中平台，页面会呈现该任务表示可执行



```
class XXXXTeamingAgent(TeamingAgentBase): new *
    def update_decision(self, new_observation: dict):...

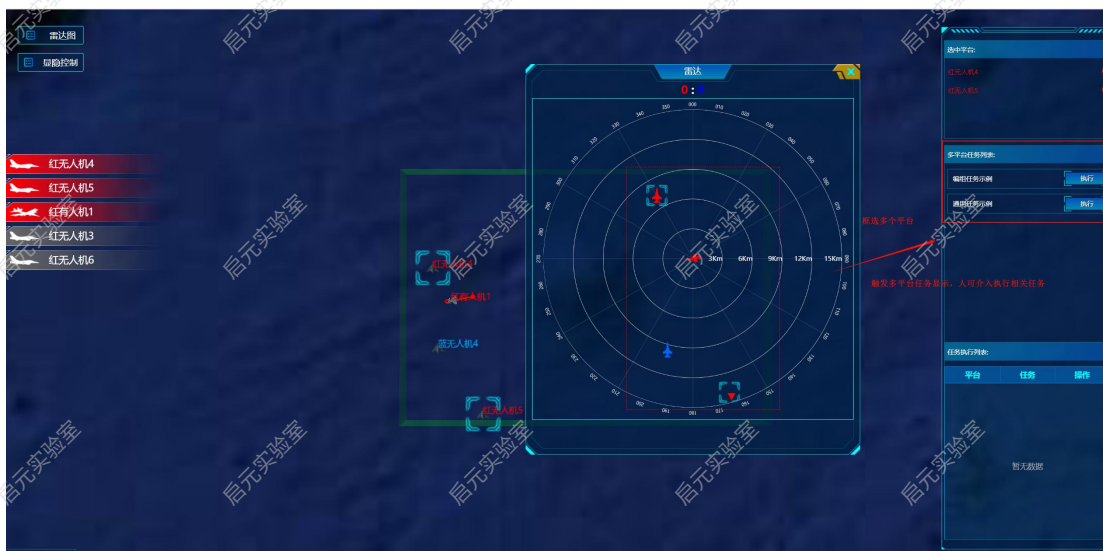
@YxRegistry.register(name="单平台任务示例", func_type=0) new *
@teaming_task
def task1(selected_forces: List[str], raw_observation: Dict):
    print(f"{selected_forces}执行单平台任务")
    actions = []
    # TODO: 单平台任务决策逻辑
    return actions func_type=0: 表示副驾驶页人员单选某平台，页面会呈现该任务表示可执行

@YxRegistry.register(name="编组任务示例", func_type=1) new *
@teaming_task
def task2(selected_forces: List[str], raw_observation: Dict):
    print(f"{selected_forces}执行编组任务")
    actions = []
    # TODO: 编组任务决策逻辑
    return actions func_type=1: 表示副驾驶页人员多选若干平台，页面会呈现该任务表示可执行

@YxRegistry.register(name="通用任务示例", func_type=2) new *
@teaming_task
def task3(selected_forces: List[str], raw_observation: Dict):
    print(f"{selected_forces}执行通用任务")
    actions = []
    # TODO: 通用任务决策逻辑
    return actions func_type=2: 表示副驾驶页人员只要选中平台，页面会呈现该任务表示可执行
```

在副驾驶页面，人员框选多个无人平台，激活了 task2 和 task3 显示在页

面上，表示人可介入执行 task2 和 task3。



4.3. 智能体开发后验证配置

开发智能体，可在 `run.py` 修改用于竞技的智能体。

在机器竞技模式下，可以将 `red_agent`、`blue_agent` 配置为自己开发的机器自主智能体，运行 AI 决策算法框架查看效果。

在人机混合编组竞技模式下，可以将 `red_agent` 或 `blue_agent` 配置为自己开发的人机协同智能体，运行 AI 决策算法框架查看效果。（目前不支持 `red_agent` 和 `blue_agent` 同时设置为人机协同智能体）。

```
run.py x
1 import ...
2
3 if __name__ == '__main__':
4
5     yxHttp.clear_room()
6     if config.current_config.battle_mode == config.ENGAGE_MODE_AUTO:
7         # 机器竞技模式
8         red_agent = DemoAutoAgent(side: 'red', name: "red_demo")
9         blue_agent = DemoAutoAgent(side: 'blue', name: "blue_demo")
10         if config.is_single_instance:
11             auto_engage_main(red_agent, blue_agent)
12         else:
13             auto_engage_main_multi(red_agent, blue_agent)
14     elif config.current_config.battle_mode == config.ENGAGE_MODE_TEAMING:
15         # 人机混合编组竞技模式
16         red_agent = DemoTeamingAgent(side: 'red', name: "red_demo")
17         blue_agent = DemoAutoAgent(side: 'blue', name: "blue_demo")
18         teaming_engage_main(red_agent, blue_agent)
```